Design Patterns

FinWallet uses design patterns only where they solve a concrete financial problem, not to accumulate pattern names. Unnecessary abstraction or interface layers should be avoided.

DDD-lite

Entities, Value Objects, Aggregates and invariants describe financial-model boundaries. The project does not use full event sourcing or heavyweight DDD infrastructure.

Adapter + Anti-Corruption Layer

Application ports such as IBankProvider, IExternalFraudProvider and ICommunicationGateway keep provider-specific DTOs/enums out of Domain. FakeBank/FakeFraud details remain in Infrastructure.

Application Orchestrator

Use cases such as registration, bank-account opening and transfer are coordinated by handlers. Controllers do not own business workflows.

Chain of Responsibility / Rule Engine

Internal fraud evaluation is composed from independent IInternalFraudRule rules. New rules can be added without substantially rewriting the transfer handler.

Policy Pattern

The combination of internal and external fraud decisions is explicitly represented through policy objects such as FraudDecisionPolicy.

Explicit Unit of Work / SQL Transaction Boundary

For wallet transfer, SqlWalletTransferPostingStore explicitly owns the transaction boundary instead of hiding it behind a generic UnitOfWork abstraction. Locking and commit order are part of business correctness.

Idempotent Command

Money-changing requests carry a client-generated Idempotency-Key. A durable MSSQL record safely replays the same key/same payload and rejects the same key/different payload.

Optimistic / Compare-and-Set Concurrency

Lifecycle updates such as BankAccount use expected status + timestamp CAS. High-contention money movement such as wallet transfer uses explicit locking/Serializable where required.

Double-Entry Ledger

Every financial effect is represented by a journal whose total Debit equals total Credit. Corrections use reversal/compensation rather than rewriting history.

State Machine

Authentication/session, BankAccount and FinancialTransaction lifecycles progress only through allowed state transitions.

Cache-Aside / TTL

Redis is used with TTL for transient state only. It is not a financial source of truth.

Timeout / Safe Retry

Provider calls have timeouts. Retries are allowed only when operation-level idempotency makes them safe; arbitrary financial POSTs are not automatically retried.

Transactional Outbox / Inbox

These are architectural plans but are not yet implemented in the production flow. They are intended for post-commit notifications and duplicate callback processing.

Saga / Compensation

This will be used only where long-running external-bank workflows justify it. A single-database wallet transfer does not use a saga because one MSSQL transaction is simpler and more correct.

Reconciliation

Ledger/balance/provider-statement mismatches are detected and investigated rather than silently corrected. Reconciliation infrastructure is planned.

CQRS-lite

Command/query code organization may be separated, but a separate read database or event-sourcing infrastructure is not required.